

Merkle tree

In cryptography and computer science, a **hash tree** or **Merkle tree** is a tree in which every "leaf" node is labelled with the cryptographic hash of a data block, and every node that is not a leaf (called a *branch*, *inner node*, or *inode*) is labelled with the cryptographic hash of the labels of its child nodes. A hash tree allows efficient and secure verification of the contents of a large data structure. A hash tree is a generalization of a hash list and a hash chain.

Demonstrating that a leaf node is a part of a given binary hash tree requires computing a number of hashes proportional to the logarithm of the number of leaf nodes in the tree.^[1] Conversely, in a hash list, the number is proportional to the number of leaf nodes itself. A Merkle tree is therefore an efficient example of a cryptographic commitment scheme, in which the root of the tree is seen as a commitment and leaf nodes may be revealed and proven to be part of the original commitment.^[2]

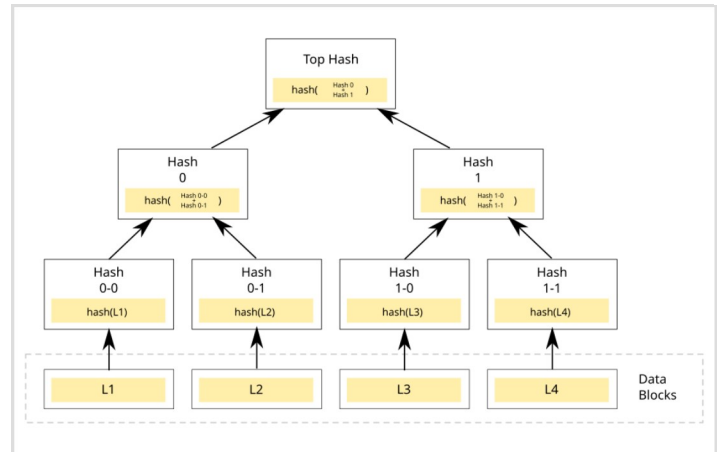
The concept of a hash tree is named after Ralph Merkle, who patented it in 1979.^{[3][4]}

Uses

Hash trees can be used to verify any kind of data stored, handled and transferred in and between computers. They can help ensure that data blocks received from other peers in a peer-to-peer network are received undamaged and unaltered, and even to check that the other peers do not lie and send fake blocks.

Hash trees are used in:

- hash-based cryptography.
- InterPlanetary File System (IPFS),
- BitTorrent
- hashtree^[5]
- ZFS file system^[6] (to counter data degradation^[7]);
- Dat protocol;
- Apache Wave protocol;^[8]



An example of a binary hash tree. Hashes 0-0 and 0-1 are the hash values of data blocks L1 and L2, respectively, and hash 0 is the hash of the concatenation of hashes 0-0 and 0-1.

- Git and Mercurial distributed revision control systems (although, strictly speaking, they use directed acyclic graphs, not trees);
- the Tahoe-LAFS backup system;
- Zeronet;
- OpenZFS^[9]
- the Bitcoin and Ethereum peer-to-peer networks;^[10]
- the Certificate Transparency framework;
- Merkle Tree Certificates;^[11]
- the Nix package manager and descendants like GNU Guix;^[12]
- a number of NoSQL systems such as Apache Cassandra, Riak, and Dynamo.^[13]

Suggestions have been made to use hash trees in trusted computing systems.^[14]

Overview

A hash tree is a tree of hashes in which the leaves (i.e., leaf nodes, sometimes also called "leafs") are hashes of data blocks in, for instance, a file or set of files. Nodes farther up in the tree are the hashes of their respective children. For example, in the above picture *hash 0* is the result of hashing the concatenation of *hash 0-0* and *hash 0-1*. That is, $hash\ 0 = hash(hash\ 0-0 + hash\ 0-1)$ where "+" denotes concatenation.

Most hash tree implementations are binary (two child nodes under each node) but they can just as well use many more child nodes under each node.

Usually, a cryptographic hash function such as SHA-2 is used for the hashing. If the hash tree only needs to protect against unintentional damage, non-cryptographic checksums such as CRCs can be used.

In the top of a hash tree there is a *top hash* (or *root hash* or *master hash*). Before downloading a file on a P2P network, in most cases the top hash is acquired from a trusted source, for instance a friend or a web site that is known to have good recommendations of files to download. When the top hash is available, the hash tree can be received from any non-trusted source, like any peer in the P2P network. Then, the received hash tree is checked against the trusted top hash, and if the hash tree is damaged or fake, another hash tree from another source will be tried until the program finds one that matches the top hash.^[15]

The main difference from a hash list is that one branch of the hash tree can be downloaded at a time and the integrity of each branch can be checked immediately, even though the whole tree is not available yet. For example, in the picture, the integrity of *data block L2* can be verified immediately if the tree already contains *hash 0-0* and *hash 1* by hashing the data block and iteratively combining the result with *hash 0-0* and then *hash 1* and finally comparing the result with the *top hash*. Similarly, the integrity of *data block L3* can be verified if the tree already has *hash 1-1* and *hash 0*. This can be an advantage since it is efficient to split files up in very small data blocks so that only small blocks have to be re-downloaded if they get damaged. If the hashed file is

big, such a hash list or hash chain becomes fairly big. But if it is a tree, one small branch can be downloaded quickly, the integrity of the branch can be checked, and then the downloading of data blocks can start.

Second preimage attack

The Merkle hash root does not indicate the tree depth, enabling a second-preimage attack in which an attacker creates a document other than the original that has the same Merkle hash root. For the example above, an attacker can create a new document containing two data blocks, where the first is *hash 0-0 + hash 0-1*, and the second is *hash 1-0 + hash 1-1*.^{[16][17]}

One simple fix is defined in Certificate Transparency: when computing leaf node hashes, a 0x00 byte is prepended to the hash data, while 0x01 is prepended when computing internal node hashes.^[15] Limiting the hash tree size is a prerequisite of some formal security proofs, and helps in making some proofs tighter. Some implementations limit the tree depth using hash tree depth prefixes before hashes, so any extracted hash chain is defined to be valid only if the prefix decreases at each step and is still positive when the leaf is reached.

Tiger tree hash

The Tiger tree hash is a widely used form of hash tree. It uses a binary hash tree (two child nodes under each node), usually has a data block size of 1024 bytes and uses the Tiger hash.^[18]

Tiger tree hashes are used in Gnutella,^[19] Gnutella2, and Direct Connect P2P file sharing protocols^[20] and in file sharing applications such as Phex,^[21] BearShare, LimeWire, Shareaza, DC++,^[22] and gtk-gnutella.^[23]

See also

- Binary tree
- Blockchain
- Distributed hash table
- Hash table
- Hash trie
- Linked timestamping
- Radix tree



References

1. Becker, Georg (2008-07-18). "Merkle Signature Schemes, Merkle Trees and Their Cryptanalysis" (https://web.archive.org/web/20141222120036/http://www.emsec.rub.de/media/crypto/attachments/files/2011/04/becker_1.pdf) (PDF). Ruhr-Universität Bochum. p. 16. Archived from the original (http://www.emsec.rub.de/media/crypto/attachments/files/2011/04/becker_1.pdf) (PDF) on 2014-12-22. Retrieved 2013-11-20.

2. "Handbook of Applied Cryptography" (<https://cacr.uwaterloo.ca/hac/>). *cacr.uwaterloo.ca*. Section 13.4.1. Retrieved 2024-03-07.
3. Merkle, R. C. (1988). "A Digital Signature Based on a Conventional Encryption Function". *Advances in Cryptology – CRYPTO '87*. Lecture Notes in Computer Science. Vol. 293. pp. 369–378. doi:10.1007/3-540-48184-2_32 (https://doi.org/10.1007%2F3-540-48184-2_32). ISBN 978-3-540-18796-7.
4. US patent 4309569 (<https://worldwide.espacenet.com/textdoc?DB=EPODOC&IDX=US4309569>), Ralph Merkle, "Method of providing digital signatures", published Jan 5, 1982, assigned to The Board of Trustees of the Leland Stanford Junior University
5. "hashtree developer page" (<https://hashtree.cc/#/dev>).
6. Bonwick, Jeff (2005-12-08). "ZFS End-to-End Data Integrity" (https://blogs.oracle.com/bonwick/entry/zfs_end_to_end_data). *blogs.oracle.com*. Archived (https://web.archive.org/web/20120403015447/https://blogs.oracle.com/bonwick/entry/zfs_end_to_end_data) from the original on April 3, 2012. Retrieved 2013-09-19.
7. Likai Liu. "Bitrot Resistance on a Single Drive" (<http://lifecs.likai.org/2014/06/bitrot-resistance-on-single-drive.html>). *likai.org*.
8. "General Verifiable Federation" (<https://web.archive.org/web/20180408150608/http://www.waveprotocol.org/whitepapers/wave-protocol-verification>). *Google Wave Protocol*. Archived from the original (<http://www.waveprotocol.org/whitepapers/wave-protocol-verification>) on 2018-04-08. Retrieved 2017-03-09.
9. "Introduction to ZFS — openzfs latest documentation" (<https://openzfs.readthedocs.io/en/latest/introduction.html>). *openzfs.readthedocs.io*. Retrieved 2025-05-27.
10. Koblitz, Neal; Menezes, Alfred J. (January 2016). "Cryptocash, cryptocurrencies, and cryptocontracts". *Designs, Codes and Cryptography*. **78** (1): 87–102. CiteSeerX 10.1.1.701.8721 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.701.8721>). doi:10.1007/s10623-015-0148-5 (<https://doi.org/10.1007%2Fs10623-015-0148-5>). S2CID 16594958 (<https://api.semanticscholar.org/CorpusID:16594958>).
11. D. Benjamin; D. O'Brien; B. E. Westerbaan; L. Valenta; F. Valsorda (2026-05-24). "Merkle Tree Certificates" (<https://datatracker.ietf.org/doc/draft-ietf-plants-merkle-tree-certs/>). *Internet Engineering Task Force*. IETF. Retrieved 2026-06-11.
12. Dolstra, E. *The Purely Functional Software Deployment Model*. (<https://nixos.org/~eelco/pubs/phd-thesis.pdf#page21>) PhD thesis, Faculty of Science, Utrecht, The Netherlands. January 2006. p.21 ISBN 90-393-4130-3.
13. Adam Marcus. "The NoSQL Ecosystem" (<http://www.aosabook.org/en/nosql.html>). *aosabook.org*. "When a replica is down for an extended period of time, or the machine storing hinted handoffs for an unavailable replica goes down as well, replicas must synchronize from one another. In this case, Cassandra and Riak implement a Dynamo-inspired process called anti-entropy. In anti-entropy, replicas exchange Merkle trees to identify parts of their replicated key ranges which are out of sync. A Merkle tree is a hierarchical hash verification: if the hash over the entire key space is not the same between two replicas, they will exchange hashes of smaller and smaller portions of the replicated key space until the out-of-sync keys are identified. This approach reduces unnecessary data transfer between replicas which contain mostly similar data."
14. Kilian, J. (1995). "Improved Efficient Arguments" (https://link.springer.com/content/pdf/10.1007/3-540-44750-4_25.pdf) (PDF). *Advances in Cryptology — CRYPTO' 95*. Lecture Notes in Computer Science. Vol. 963. pp. 311–324. doi:10.1007/3-540-44750-4_25 (https://doi.org/10.1007%2F3-540-44750-4_25). ISBN 978-3-540-60221-7.

15. Laurie, B.; Langley, A.; Kasper, E. (June 2013). "Certificate Transparency" (<https://www.rfc-editor.org/info/rfc6962>). *IETF RFC6962*. doi:10.17487/rfc6962 (<https://doi.org/10.17487%2Frfc6962>).
16. Elena Andreeva; Charles Bouillaguet; Orr Dunkelman; John Kelsey (January 2009). "Herding, Second Preimage and Trojan Message Attacks beyond Merkle-Damgård". *Selected Areas in Cryptography*. Lecture Notes in Computer Science. Vol. 5867. SAC. pp. 393–414. doi:10.1007/978-3-642-05445-7_25 (https://doi.org/10.1007%2F978-3-642-05445-7_25). ISBN 978-3-642-05443-3.
17. Elena Andreeva; Charles Bouillaguet; Pierre-Alain Fouque; Jonathan J. Hoch; John Kelsey; Adi Shamir; Sebastien Zimmer (2008). "Second Preimage Attacks on Dithered Hash Functions". In Smart, Nigel (ed.). *Advances in Cryptology – EUROCRYPT 2008*. Lecture Notes in Computer Science. Vol. 4965. Istanbul, Turkey. pp. 270–288. doi:10.1007/978-3-540-78967-3_16 (https://doi.org/10.1007%2F978-3-540-78967-3_16). ISBN 978-3-540-78966-6. S2CID 12844017 (<https://api.semanticscholar.org/CorpusID:12844017>).
18. Chapweske, J.; Mohr, G. (March 4, 2003). "Tree Hash EXchange format (THEX)" (<https://web.archive.org/web/20090803220648/http://open-content.net/specs/draft-jchapweske-thex-02.html>). Archived from the original on 2009-08-03.
19. "tigertree.c File Reference" (https://gtk-gnutella.sourceforge.net/doxygen/tigertree_8c.html). *Gtk-Gnutella*. Retrieved 23 September 2018.
20. "Audit: P2P DirectConnect Application" (https://web.archive.org/web/20150129050053/http://www.symantec.com/security_response/attacksignatures/detail.jsp?asid=21587). *Symantec*. Archived from the original (https://www.symantec.com/security_response/attacksignatures/detail.jsp?asid=21587) on January 29, 2015. Retrieved 23 September 2018.
21. Arne Babenhauserheide (7 Jan 2007). "Phex 3.0.0 released" (<http://www.phex.org/mambo/content/view/80/2/>). *Phex*. Retrieved 23 September 2018.
22. "DC++'s feature list" (<http://dcplusplus.sourceforge.net/features.html>). *dcplusplus.sourceforge.net*.
23. "Development" (<http://gtk-gnutella.sourceforge.net/en/?page=devel>). *GTK-Gnutella*. Retrieved 23 September 2018.

Further reading

- Merkle tree patent 4,309,569 (<https://patents.google.com/patent/US4309569>) – explains both the hash tree structure and the use of it to handle many one-time signatures
- Tree Hash EXchange format (THEX) (<https://web.archive.org/web/20080316033726/http://www.open-content.net/specs/draft-jchapweske-thex-02.html>) – a detailed description of Tiger trees

External links

- A C implementation of a dynamically re-sizeable binary SHA-256 hash tree (Merkle tree) (<https://github.com/IAIK/merkle-tree>)
- Merkle tree implementation in Java (<https://github.com/richpl/merkletree>)
- Tiger Tree Hash (TTH) source code in C# (<https://www.codeproject.com/articles/9336/thexcs-tt-h-tiger-tree-hash-maker-in-c>), by Gil Schmidt
- Tiger Tree Hash (TTH) implementations in C and Java (<https://sourceforge.net/projects/tigertre>)

e/)

- RHash (<https://rhash.sourceforge.net/>), an open source command-line tool, which can calculate TTH and magnet links with TTH

Retrieved from "https://en.wikipedia.org/w/index.php?title=Merkle_tree&oldid=1359303374"